Vishal Acharya

Streamlit

streamlit run app.py

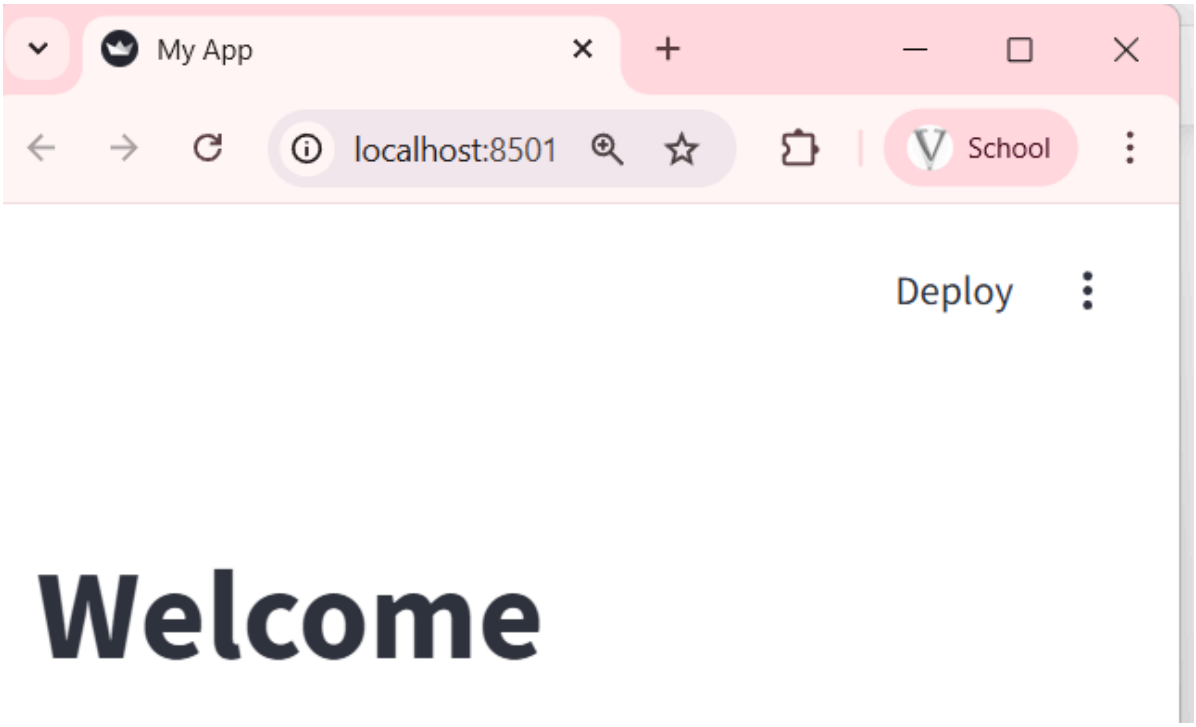
- for execute the code

TITLE: Page Configuration

- DEFINITION: st.set_page_config() is used to set basic configuration of a Streamlit app such as page title, layout, and icon.
- PURPOSE: To control the appearance and layout of the Streamlit application page.
- SYNTAX:
- st.set_page_config(page_title="Title", layout="wide")

In [1]:
```
%%writefile app.py
import streamlit as st
st.set_page_config(page_title="My App", layout="wide")
st.title("Welcome")
```

```
Writing app.py
```



In [4]:
```
%%writefile app.py
import streamlit as st
st.set_page_config(page_title="My App", layout="centered")
st.title("Welcome")
```

```
Overwriting app.py
```

Feature	`layout="centered"`	`layout="wide"`
Screen usage	Uses a **fixed-width, centered** content area	Uses **full browser width**
Content position	Content stays in the **middle** of the page	Content stretches **left to right**
Best for	Simple apps, forms, text-based pages	Dashboards, tables, charts, multiple columns
Columns	Limited space for many columns	Ideal for **multiple columns & charts**
Default behavior	**Default layout** in Streamlit	Must be explicitly set

Vishal Acharya

2. Text Elements (st.title, st.header, st.subheader, st.text, st.write, st.markdown, st.code)

- TITLE: Text Elements
- DEFINITION: Text elements are used to display headings, text, formatted content, and code in a Streamlit app.
- PURPOSE: To present information clearly and structurally to the user.
- SYNTAX:

1. st.title(text)
2. st.header(text)
3. st.subheader(text)
4. st.text(text)
5. st.write(object)
6. st.markdown(markdown_text)
7. st.code(code, language)

Style rules

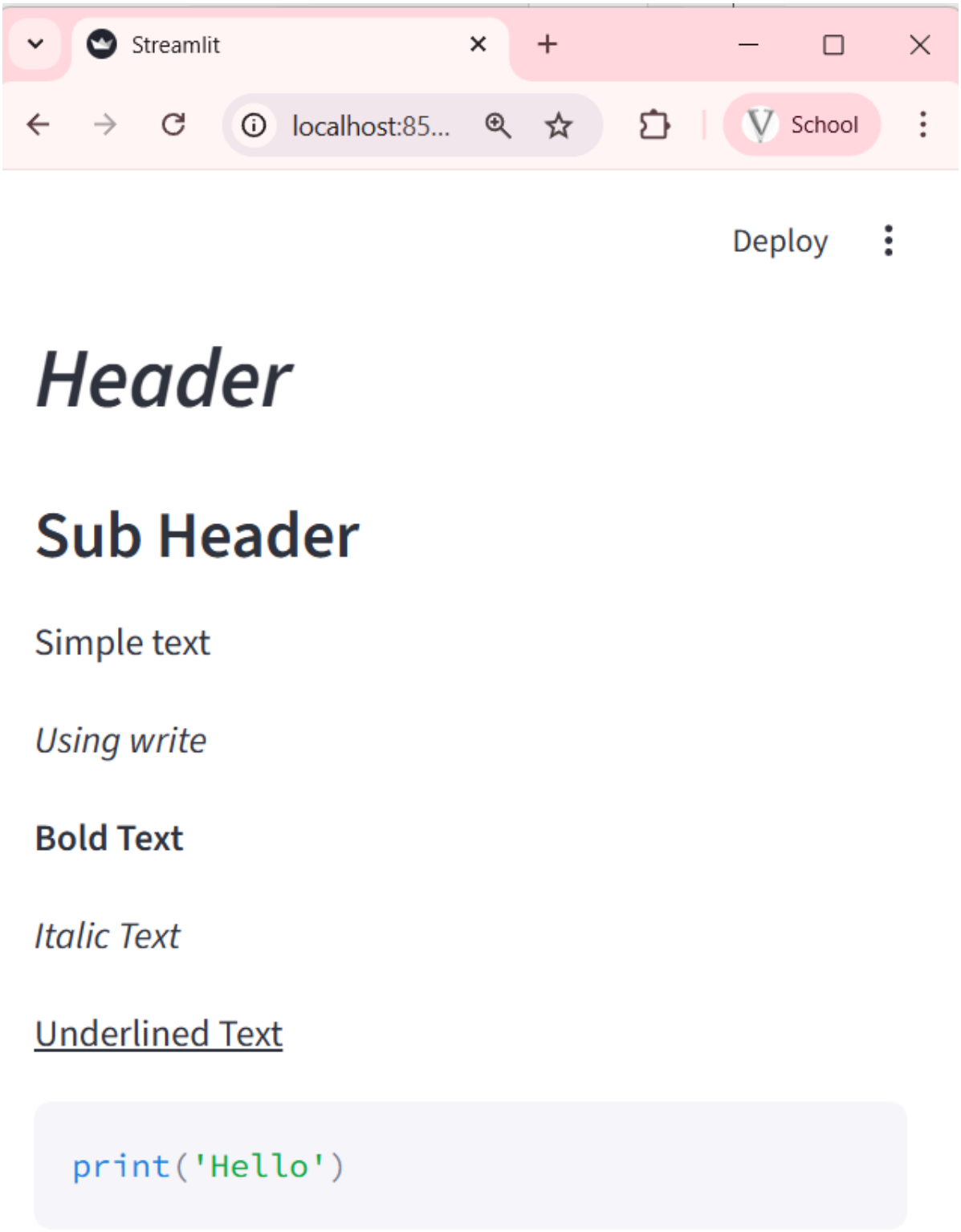
```
- Bold → **text**

- Italic → _text_ or *text*

- <u>Underline</u> → HTML tag with unsafe_allow_html=True

- st.text() → No styling supported

- st.write() → Supports basic Markdown
```

In [6]:

```
%%writefile app.py
import streamlit as st
st.title("Streamlit App")

st.header("_Header_")                # Italic
st.subheader("__Sub Header__")       # Underline

st.text("Simple text")
```

```
st.write("*Using write*")             # Italic

st.markdown("**Bold Text**")          # Bold
st.markdown("_Italic Text_")          # Italic
st.markdown("<u>Underlined Text</u>", unsafe_allow_html=True)  # Underline

st.code("print('Hello')", language="python")
```

```
Overwriting app.py
```



Vishal Acharya

3. Layout Components (st.sidebar, st.columns, st.expander)

- TITLE: Layout Components

- DEFINITION: Layout components help organize content using sidebar, columns, and expandable sections.
- PURPOSE: To improve page structure and user interaction.
- SYNTAX:

1. st.sidebar.widget(): is a special container in Streamlit used to place widgets on the left side of the application.

- To separate controls (inputs, filters, menus) from the main content
- To improve app layout and usability

2. st.columns(n):creates multiple columns in the main page to display content side by side.

- To organize content horizontally
- To display charts, text, or inputs in parallel

OUTPUT

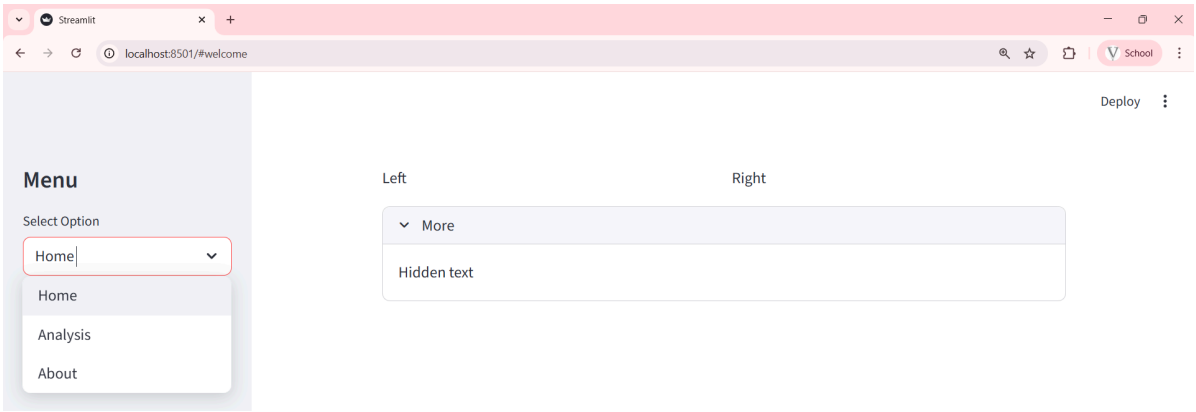
- The page is divided into two vertical columns
- Each column displays its own content

3. st.expander(label): creates a collapsible section that can hide or show content.

- To keep the interface clean
- To display optional or detailed information only when needed
- Content is shown only when the user expands it

In [9]:
```
%%writefile app.py
import streamlit as st
st.sidebar.title("Menu")
option = st.sidebar.selectbox(
    "Select Option",
    ["Home", "Analysis", "About"]
)
col1, col2 = st.columns(2)
col1.write("Left")
col2.write("Right")
with st.expander("More"):
    st.write("Hidden text")
```

```
Overwriting app.py
```



4. Text Inputs (st.text_input, st.text_area)

Vishal Acharya

- TITLE: Text Input Widgets
- DEFINITION: Text input widgets allow users to enter single-line or multi-line text.
- PURPOSE: To collect text data from users.
- SYNTAX:

1. st.text_input(label)

- To accept short text such as name, username, email, or ID
- To capture one line of input only

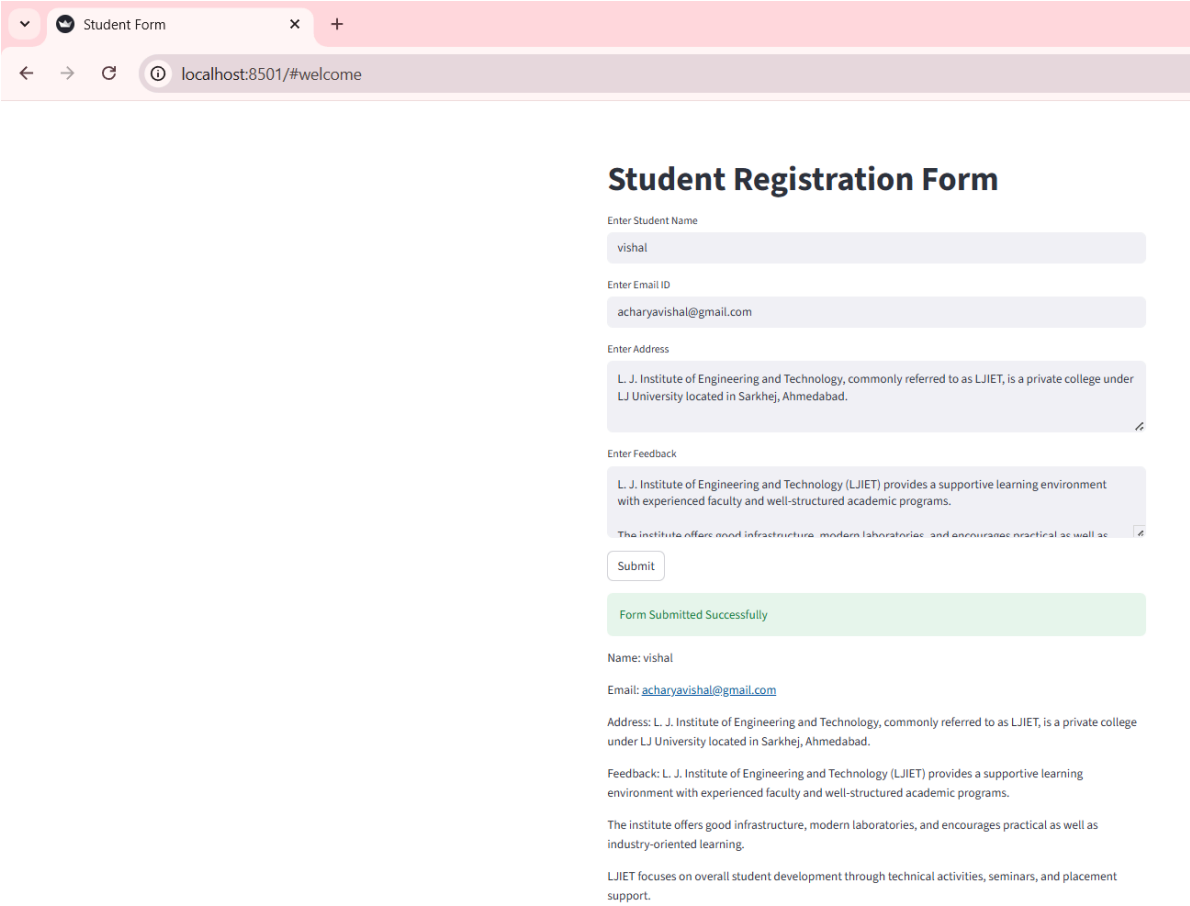
2. st.text_area(label)

- To accept long text such as address, feedback, or comments
- To allow multiple lines of input

In [10]:
```
%%writefile app.py
import streamlit as st

st.set_page_config(page_title="Student Form", layout="centered")

st.title("Student Registration Form")

# Text Input (Single-line)
name = st.text_input("Enter Student Name")

email = st.text_input("Enter Email ID")

# Text Area (Multi-line)
address = st.text_area("Enter Address")

feedback = st.text_area("Enter Feedback")

# Submit Button
if st.button("Submit"):
    if name and email and address:
        st.success("Form Submitted Successfully")
        st.write("Name:", name)
        st.write("Email:", email)
        st.write("Address:", address)
        st.write("Feedback:", feedback)
    else:
        st.warning("Please fill all required fields")
```

```
Overwriting app.py
```

Vishal Acharya



Vishal Acharya

5. Number Inputs (st.number_input, st.slider)

- TITLE: Number Input Widgets
- DEFINITION: These widgets allow users to input numeric values.
- PURPOSE: To accept numerical input from users.
- SYNTAX:

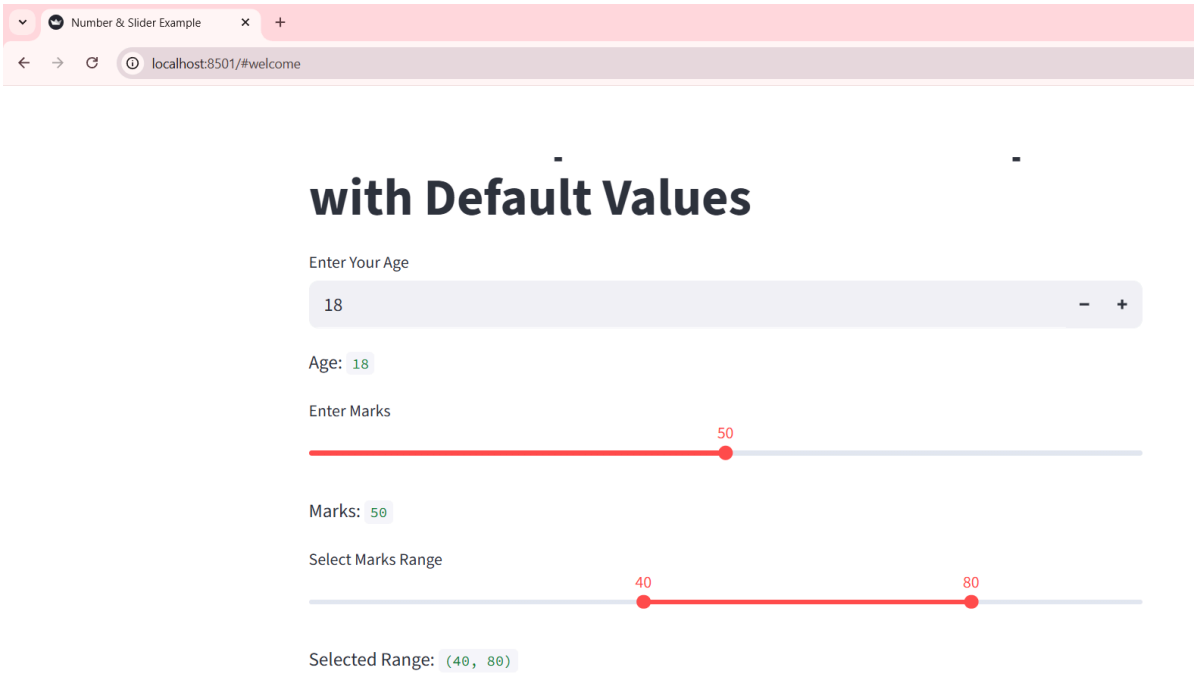
1. st.number_input(label)
2. st.slider(label, min, max,defalut_value)

Feature	st.number_input()	st.slider()
Input method	Type number / arrows	Drag slider
Precision	High (exact value)	Moderate
Best for	Age, quantity	Marks, rating
UI	Input box	Slider bar

In [11]:
```
%%writefile app.py
import streamlit as st

st.set_page_config(page_title="Number & Slider Example", layout="centered")
st.title("Number Input and Slider Example with Default Values")

# ---------------- Number Input ----------------
age = st.number_input(
    "Enter Your Age",
    min_value=0,
```

```
    max_value=100,
    value=18            # Default value
)
st.write("Age:", age)

# ---------------- Single Slider ----------------
marks = st.slider(
    "Enter Marks",
    min_value=0,
    max_value=100,
    value=50            # Default value
)
st.write("Marks:", marks)

# ---------------- Range Slider ----------------
range_marks = st.slider(
    "Select Marks Range",
    min_value=0,
    max_value=100,
    value=(40, 80)      # Default range
)
st.write("Selected Range:", range_marks)
```

```
Overwriting app.py
```



6. Selection Widgets (st.selectbox, st.multiselect, st.radio, st.checkbox)

- TITLE: Selection Widgets
- DEFINITION: Selection widgets allow users to choose one or more options.
- PURPOSE: To take categorical input from users.
- SYNTAX:

1. st.selectbox(label, options): creates a dropdown menu from which the user can select one option.

- To take single-choice categorical input from the user.

Vishal Acharya

- Useful for selecting city, country, department, etc.

2. st.multiselect(label, options): allows the user to select multiple options from a list.

- To take multiple-choice input.
- Useful for selecting multiple skills, subjects, or interests.

3. st.radio(label, options): allows the user to select one option from a radio button list.

- For single-choice input where all options are visible at once.
- Useful for selecting gender, yes/no, payment method.

4. st.checkbox(label): provides a tick-box for boolean input (True/False).

- To take Yes/No or Agree/Disagree input.
- Often used for terms & conditions or optional features.

Widget	Input Type	Multiple Selection	Visible Options
selectbox	Single	❌	Dropdown
multiselect	Multiple	✅	Dropdown
radio	Single	❌	All visible
checkbox	Boolean	❌	Single tick

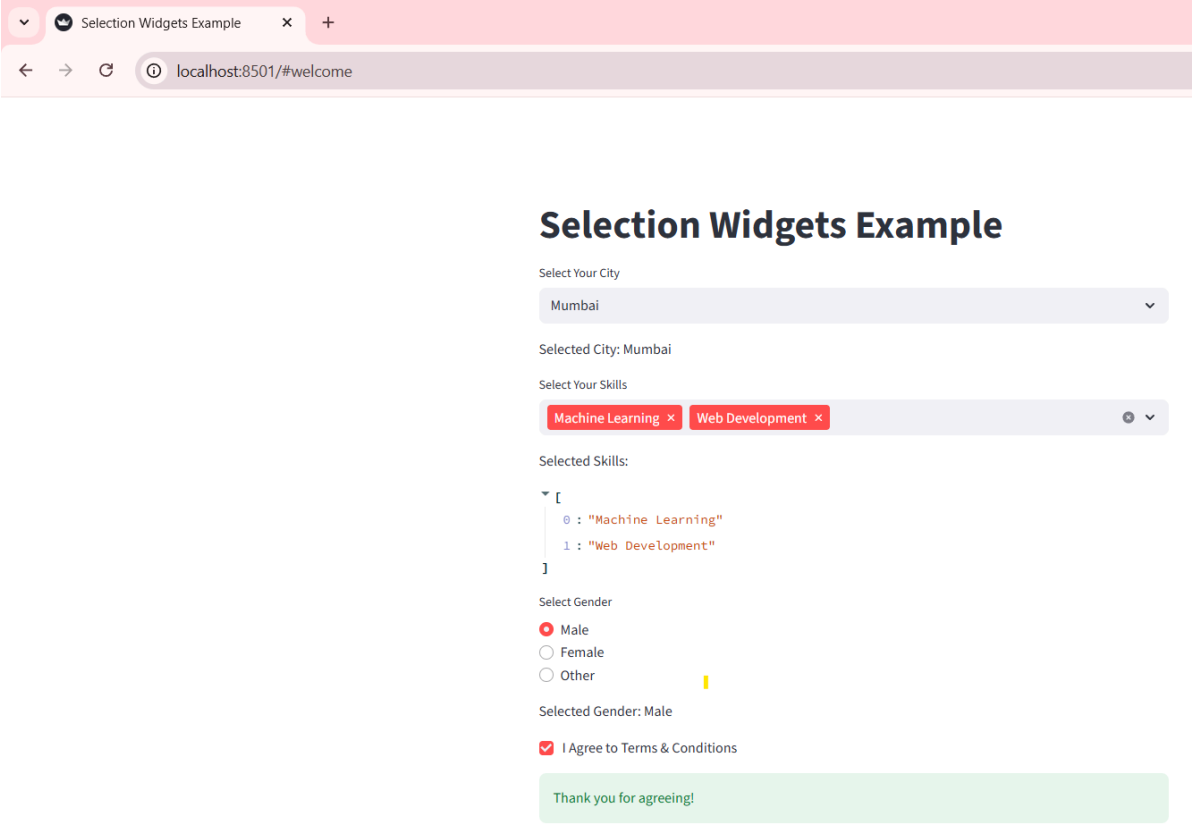
In [12]:

```
%%writefile app.py
import streamlit as st

st.set_page_config(page_title="Selection Widgets Example", layout="centered")
st.title("Selection Widgets Example")

# ---------------- Selectbox (Single Choice) ----------------
city = st.selectbox(
    "Select Your City",
    ["Ahmedabad", "Mumbai", "Delhi", "Bangalore"]
)
st.write("Selected City:", city)

# ---------------- Multiselect (Multiple Choice) ----------------
skills = st.multiselect(
    "Select Your Skills",
    ["Python", "Machine Learning", "Data Science", "Web Development"]
)
st.write("Selected Skills:", skills)

# ---------------- Radio Buttons (Single Choice) ----------------
gender = st.radio(
    "Select Gender",
    ["Male", "Female", "Other"]
)
st.write("Selected Gender:", gender)

# ---------------- Checkbox (Boolean Input) ----------------
agree = st.checkbox("I Agree to Terms & Conditions")
if agree:
    st.success("Thank you for agreeing!")
```

Vishal Acharya

```
else:
    st.info("Please agree to continue.")
```

```
Overwriting app.py
```



7. Date & Time Inputs (st.date_input, st.time_input)

- TITLE: Date and Time Inputs
- DEFINITION: These widgets allow users to select date and time.
- PURPOSE: To capture date and time values.
- SYNTAX:

1. st.date_input(label)

- st.date_input(label, value=None, min_value=None, max_value=None)
- To input dates such as birth date, appointment date, or event date.
- Provides a user-friendly calendar widget.

2. st.time_input(label)

- st.time_input(label, value=None)
- To input time for appointments, events, or schedules.
- Provides a user-friendly time picker

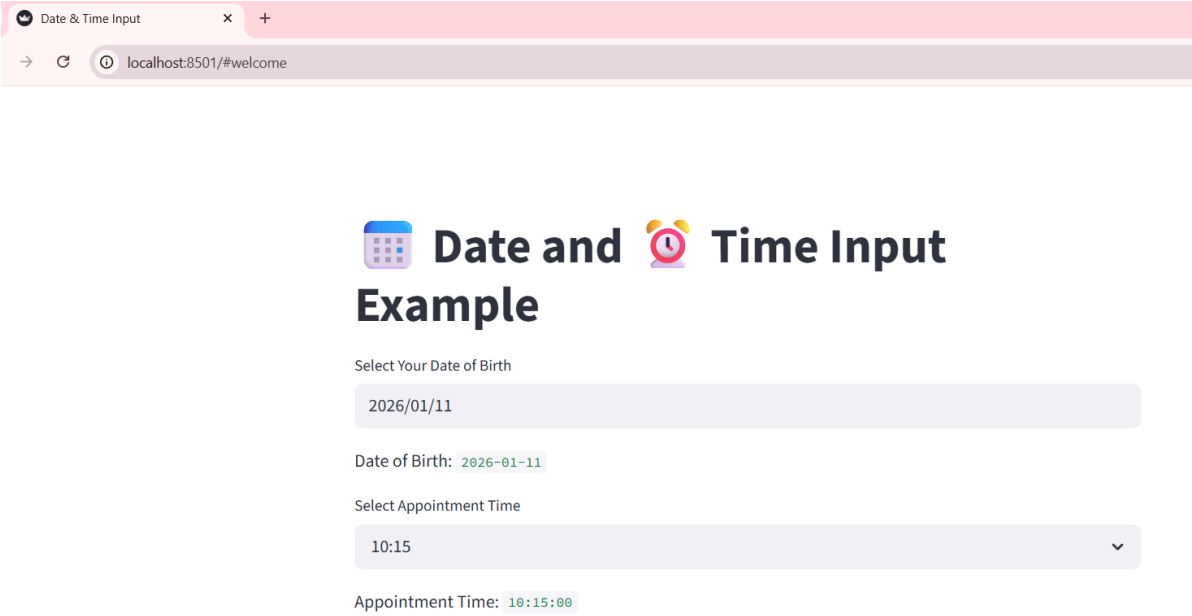
In [15]:

```
%%writefile app.py
import streamlit as st
from datetime import date, time

st.set_page_config(page_title="Date & Time Input", layout="centered")
st.title("📅 Date and ⏰ Time Input Example")

# ---------------- Date Input ----------------
```

Vishal Acharya

```
dob = st.date_input(
    "Select Your Date of Birth",
    value=date(2026, 1, 1)   # Default value
)
st.write("Date of Birth:", dob)

# ---------------- Time Input ----------------
appt_time = st.time_input(
    "Select Appointment Time",
    value=time(9, 30)        # Default value
)
st.write("Appointment Time:", appt_time)
```

```
Overwriting app.py
```



st.time_input(label, value=None, step=timedelta(minutes=1))

value → default time

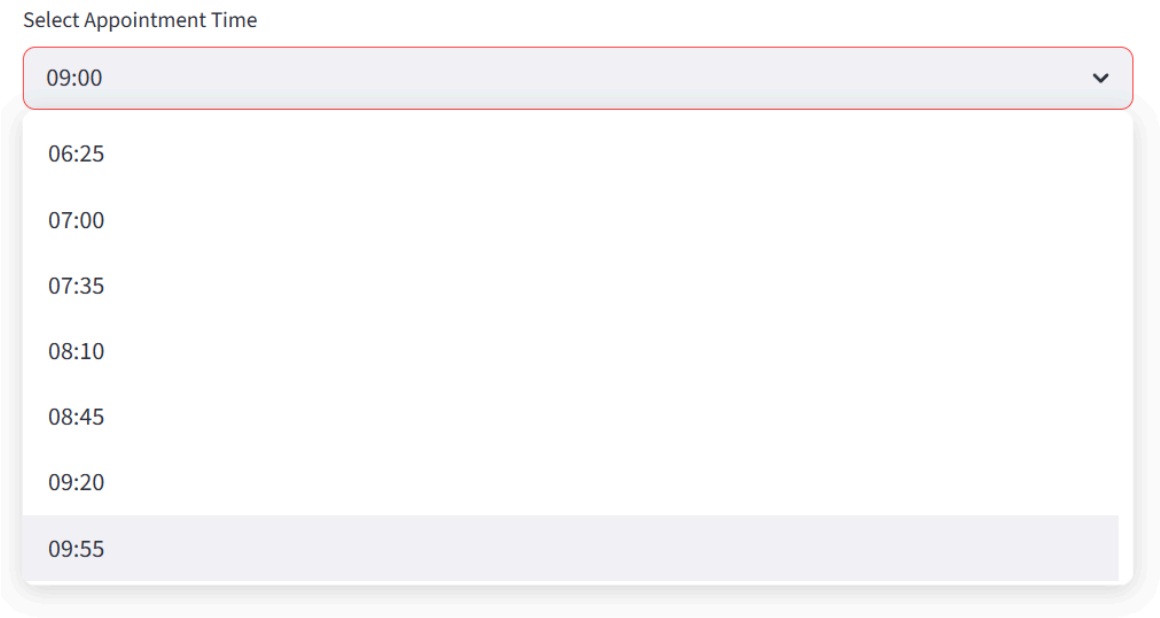
step → time increment as datetime.timedelta

In [16]:

```
%%writefile app.py
import streamlit as st
from datetime import time, timedelta

appt_time = st.time_input(
    "Select Appointment Time",
    value=time(9, 0),
    step=timedelta(minutes=35)   # increments of 15 minutes
)
st.write("Appointment Time:", appt_time)
```

```
Overwriting app.py
```

Vishal Acharya



Vishal Acharya

8. File Uploader (st.file_uploader)

- TITLE: File Uploader
- DEFINITION: Used to upload files through the Streamlit interface.
- PURPOSE: To allow users to upload external files.
- SYNTAX:

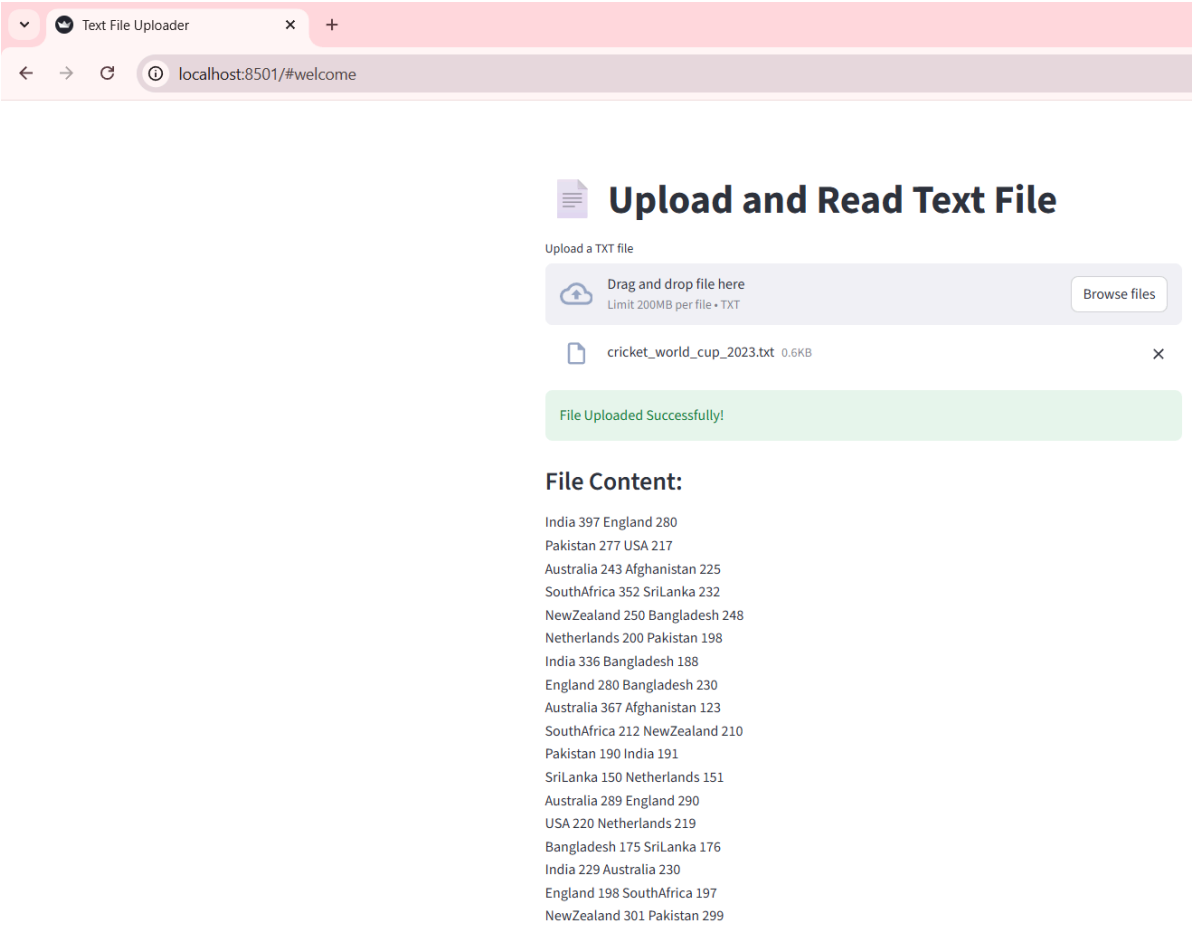
1. st.file_uploader(label)

- st.file_uploader(label, type=None, accept_multiple_files=False)
- label → Name shown above the uploader.
- type → List of allowed file types, e.g., ["csv", "txt", "xlsx"].
- accept_multiple_files → Set True to allow multiple file uploads.
- To accept files such as CSV, Excel, images, PDF, or text for processing.
- Commonly used in data analysis apps, image processing apps, and dashboards.

In [17]:
```
%%writefile app.py
import streamlit as st

st.set_page_config(page_title="Text File Uploader", layout="centered")
st.title("📄 Upload and Read Text File")

# File uploader for TXT files
uploaded_file = st.file_uploader("Upload a TXT file", type=["txt"])

if uploaded_file is not None:
    # Read file content
    content = uploaded_file.read().decode("utf-8")

    st.success("File Uploaded Successfully!")
```

```
    st.subheader("File Content:")
    st.text(content)
```

```
Overwriting app.py
```



Vishal Acharya

9. Status Elements (st.progress, st.spinner, st.success, st.error, st.warning, st.info)

- TITLE: Status Elements
- DEFINITION: Status elements show progress, loading, and messages to users.
- PURPOSE: To give feedback about system status.
- SYNTAX:

1. st.progress(value): Displays a progress bar in the app

- To show the progress of a long-running task.
- Improves user experience by giving visual feedback

2. st.spinner(text): Displays a loading spinner while a task is running.

- To indicate that the app is processing.
- Provides feedback during time-consuming operations.
- Shows a spinner with text "Processing..." for 3 seconds.
- After completion, shows success message.

3. st.success(text):Displays a green success message.

- To indicate successful completion of a task or action.

4. st.error(text): Displays a red error message.

- To indicate an error or failure in the app or input.

5. st.warning(text):Displays a yellow warning message.

- To alert the user about potential issues or cautions.

6. st.info(text): Displays a blue informational message.

- To provide informational messages or guidance to users.

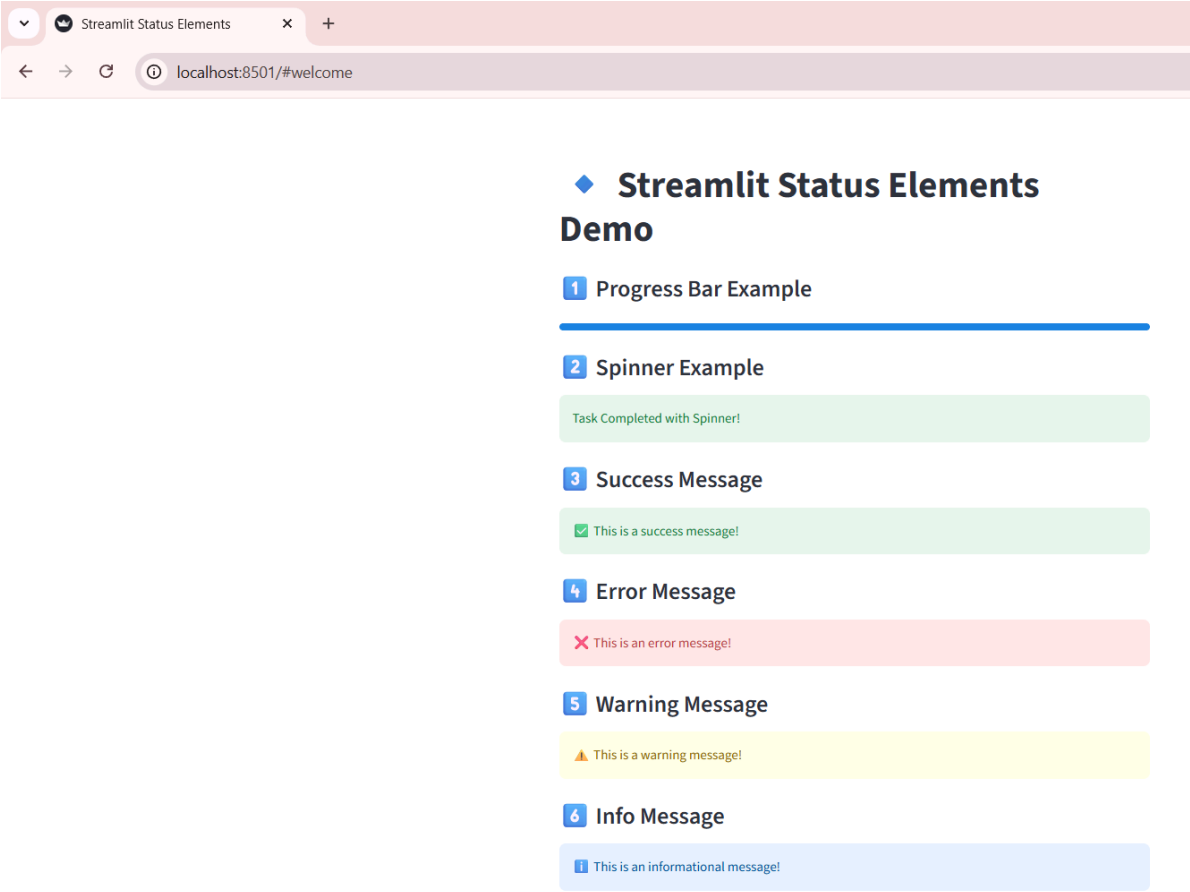
Widget	Color	Purpose
`st.progress`	Blue (bar)	Show progress of a task
`st.spinner`	Animated	Indicate processing
`st.success`	Green	Task completed successfully
`st.error`	Red	Task failed / error
`st.warning`	Yellow	Alert / caution
`st.info`	Blue	Informational message

Vishal Acharya

In [18]:

```
%%writefile app.py
import streamlit as st
import time

st.set_page_config(page_title="Streamlit Status Elements", layout="centered")
st.title("🔹 Streamlit Status Elements Demo")

st.subheader("1️⃣ Progress Bar Example")
my_bar = st.progress(0)
for i in range(101):
    time.sleep(0.01)  # Simulate work
    my_bar.progress(i)

st.subheader("2️⃣ Spinner Example")
with st.spinner("Processing task..."):
    time.sleep(2)  # Simulate a long task
st.success("Task Completed with Spinner!")

st.subheader("3️⃣ Success Message")
st.success("✅ This is a success message!")

st.subheader("4️⃣ Error Message")
st.error("❌ This is an error message!")

st.subheader("5️⃣ Warning Message")
st.warning("⚠️ This is a warning message!")

st.subheader("6️⃣ Info Message")
st.info("ℹ️ This is an informational message!")
```

```
Overwriting app.py
```



10. Matplotlib Charts (Line, Bar, Scatter, Histogram)

- TITLE: Matplotlib Charts
- DEFINITION: Matplotlib charts are visual graphs displayed in Streamlit using st.pyplot().
- PURPOSE: To visualize data graphically.
- SYNTAX:
- plt.plot(x, y)
- st.pyplot(plt)

In [24]:
```
%%writefile app.py
import streamlit as st
import matplotlib.pyplot as plt

st.set_page_config(page_title="Matplotlib Charts", layout="wide")
st.title("📊 Matplotlib Charts in Streamlit (Reduced Size)")

# ---------------- Line Chart ----------------
st.subheader("1️⃣ Line Chart")
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]

plt.figure(figsize=(2,1))
plt.plot(x, y, marker='o', linestyle='-', color='blue')
plt.title("Line Chart Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
st.pyplot(plt)

# ---------------- Bar Chart ----------------
st.subheader("2️⃣ Bar Chart")
categories = ['A', 'B', 'C', 'D']
```

Vishal Acharya

Vishal Acharya

```
values = [10, 20, 15, 25]

plt.figure(figsize=(2,1))
plt.bar(categories, values, color='green')
plt.title("Bar Chart Example")
st.pyplot(plt)

# ---------------- Scatter Plot ----------------
st.subheader("3️⃣ Scatter Plot")
x_scatter = [5, 7, 8, 7, 2, 17]
y_scatter = [99, 86, 87, 88, 100, 86]

plt.figure(figsize=(2,1))
plt.scatter(x_scatter, y_scatter, color='red')
plt.title("Scatter Plot Example")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
st.pyplot(plt)

# ---------------- Histogram ----------------
st.subheader("4️⃣ Histogram")
data = [22, 55, 62, 45, 21, 22, 34, 42, 42, 60]

plt.figure(figsize=(2,1))
plt.hist(data, bins=5, color='orange', edgecolor='black')
plt.title("Histogram Example")
plt.xlabel("Value")
plt.ylabel("Frequency")
st.pyplot(plt)
```

```
Overwriting app.py
```

- for output run file
- streamlit run app.py

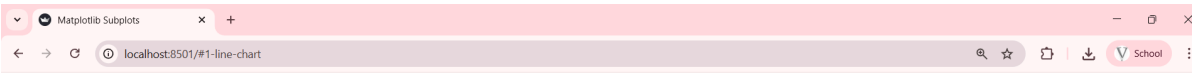
In [21]:

```
%%writefile app.py
import streamlit as st
import matplotlib.pyplot as plt

st.set_page_config(page_title="Matplotlib Subplots", layout="wide")
st.title("📊 Matplotlib Charts in Streamlit (Subplots without axes array)")

plt.figure(figsize=(20,5))

# ---------------- Line Chart ----------------
plt.subplot(1, 4, 1)  # 1 row, 4 columns, first plot
x = [1, 2, 3, 4, 5]
y = [10, 20, 15, 25, 30]
plt.plot(x, y, marker='o', linestyle='-', color='blue')
plt.title("Line Chart")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")

# ---------------- Bar Chart ----------------
plt.subplot(1, 4, 2)  # 1 row, 4 columns, second plot
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]
plt.bar(categories, values, color='green')
plt.title("Bar Chart")

# ---------------- Scatter Plot ----------------
```

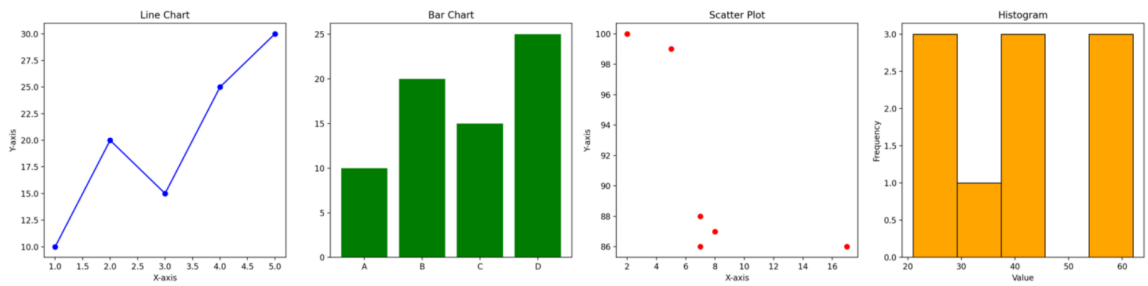
Vishal Acharya

```
plt.subplot(1, 4, 3)  # 1 row, 4 columns, third plot
x_scatter = [5, 7, 8, 7, 2, 17]
y_scatter = [99, 86, 87, 88, 100, 86]
plt.scatter(x_scatter, y_scatter, color='red')
plt.title("Scatter Plot")
plt.xlabel("X-axis")
plt.ylabel("Y-axis")

# ---------------- Histogram ----------------
plt.subplot(1, 4, 4)  # 1 row, 4 columns, fourth plot
data = [22, 55, 62, 45, 21, 22, 34, 42, 42, 60]
plt.hist(data, bins=5, color='orange', edgecolor='black')
plt.title("Histogram")
plt.xlabel("Value")
plt.ylabel("Frequency")

# Adjust layout
plt.tight_layout()

# Display in Streamlit
st.pyplot(plt)
```

```
Overwriting app.py
```



📊 Matplotlib Charts in Streamlit (Subplots without axes array)



11. Streamlit Built-in Charts (st.line_chart, st.bar_chart, st.area_chart)

- TITLE: Built-in Charts
- DEFINITION: Built-in charts provide quick visualization without Matplotlib.
- PURPOSE: To quickly plot data with minimal code.
- SYNTAX:

1. st.line_chart(data): displays data as a line graph using Streamlit's built-in charting.

- To show trends over time or sequence
- Quick visualization without using Matplotlib

2. st.bar_chart(data): displays data using vertical bars.

- To compare values across categories
- Simple bar visualization without Matplotlib

3. st.area_chart(data):displays data as a filled area under a line.

- To show cumulative trends
- Useful when comparing growth patterns

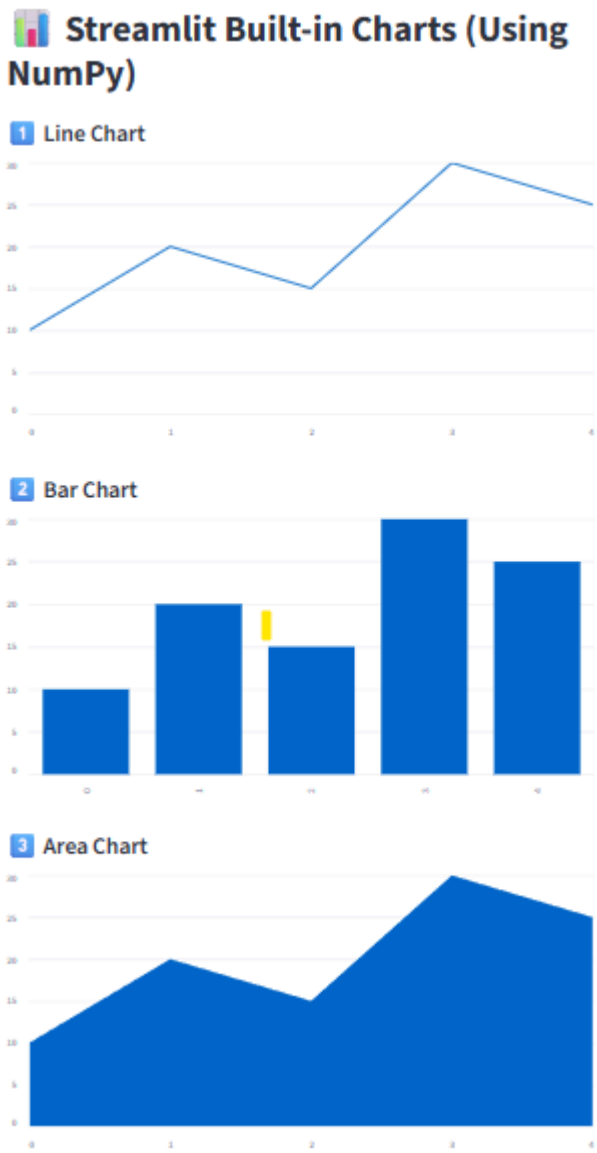
- Chart Type Best Use
- st.line_chart Trend over time
- st.bar_chart Category comparison
- st.area_chart Growth & cumulative data

In [25]:
```
%%writefile app.py
import streamlit as st
import numpy as np

st.set_page_config(page_title="Built-in Charts with NumPy", layout="centered")
st.title("📊 Streamlit Built-in Charts (Using NumPy)")

# NumPy Array Data
data = np.array([10, 20, 15, 30, 25])

# ---------------- Line Chart ----------------
st.subheader("1️⃣ Line Chart")
st.line_chart(data)

# ---------------- Bar Chart ----------------
st.subheader("2️⃣ Bar Chart")
st.bar_chart(data)

# ---------------- Area Chart ----------------
st.subheader("3️⃣ Area Chart")
st.area_chart(data)
```

```
Overwriting app.py
```

Vishal Acharya

Vishal Acharya



st.dataframe

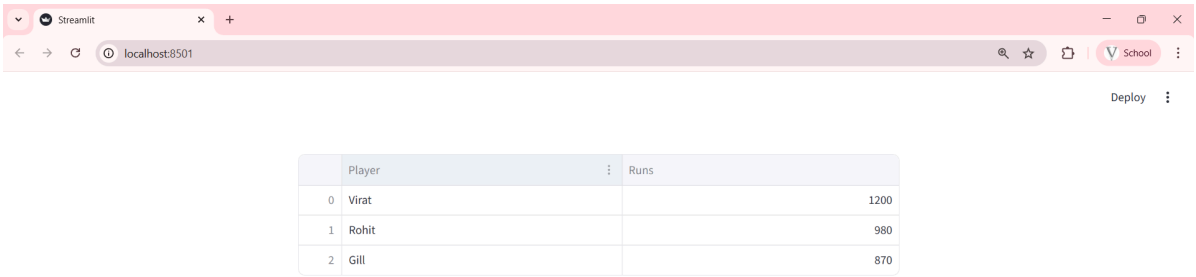
- Title: st.dataframe – Display Interactive Data Table
- Syntax: st.dataframe(data)
- Definition: st.dataframe is used to display tabular data (DataFrame, list, NumPy array) in an interactive table where users can scroll, resize columns, and sort data.
- Purpose

1. Show large datasets
2. Allow scrolling and column resizing
3. Best for data analysis dashboards

In [1]:
```
%%writefile app.py
import streamlit as st
import pandas as pd

data = {
```

```
    "Player": ["Virat", "Rohit", "Gill"],
    "Runs": [1200, 980, 870]
}

df = pd.DataFrame(data)

st.dataframe(df)
```

```
Overwriting app.py
```



st.table

- Title: st.table – Display Static Table
- Syntax:st.table(data)
- Definition: st.table displays data in a fixed, non-interactive table format.
- Purpose

1. Display small, clean tables
2. No scrolling or sorting
3. Best for summary data

In [3]:
```
%%writefile app.py
import streamlit as st
marks = {
    "Subject": ["Maths", "Science", "English"],
    "Marks": [85, 90, 88]
}
st.table(marks)
```

```
Overwriting app.py
```

st.json

- Title: st.json – Display JSON Data
- Syntax: st.json(data)

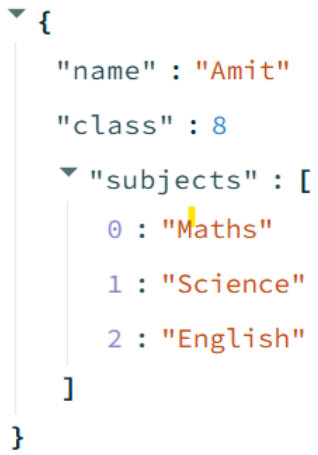
Vishal Acharya

- Definition: st.json displays JSON or dictionary data in a pretty formatted, expandable view.
- Purpose:

1. Show API responses
2. Debug JSON data
3. Easy to read nested data

In [5]:
```
%%writefile app.py
import streamlit as st

student = {
    "name": "Amit",
    "class": 8,
    "subjects": ["Maths", "Science", "English"]
}

st.json(student)
```

```
Overwriting app.py
```



st.image

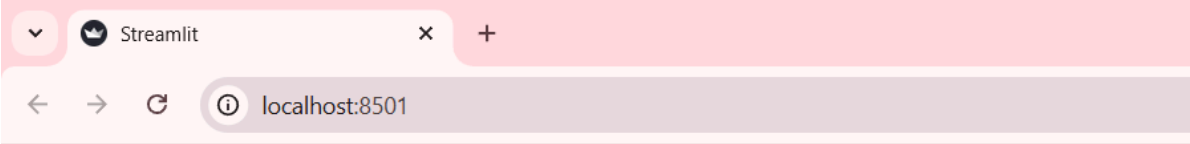
- Title: st.image – Display Image
- Syntax: st.image(image, caption=None)
- Definition: st.image displays images in formats like JPG, PNG from file or URL.
- Purpose:

1. Show logos
2. Display charts as images
3. Add visual content

Vishal Acharya

In [6]:
```
%%writefile app.py
import streamlit as st

st.image("logo.png", caption="College Logo")
```

```
Overwriting app.py
```



College Logo

Vishal Acharya

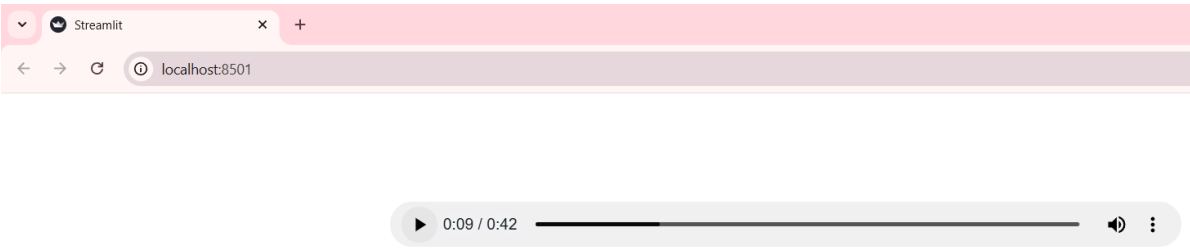
st.audio

- Title: st.audio – Play Audio File
- Syntax: st.audio(audio_file)
- Definition: st.audio is used to play audio files such as MP3 or WAV.
- Purpose

1. Play speeches
2. Music samples
3. Voice recordings

In [8]:
```
%%writefile app.py
import streamlit as st
audio_file = open("song.mp3", "rb")
st.audio(audio_file)
```

```
Overwriting app.py
```



Vishal Acharya

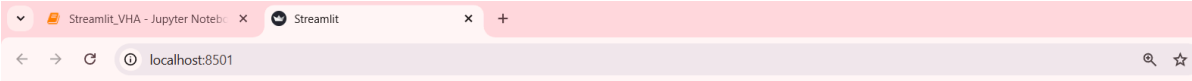
st.video

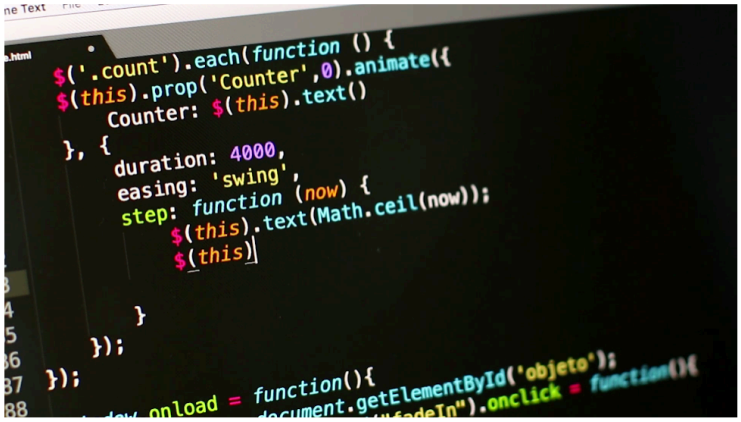
- Title: st.video – Play Video
- Syntax: st.video(video_file)
- Definition: st.video is used to play video files or YouTube links.
- Purpose:

1. Show tutorials
2. Project demos: Educational videos

In [11]:
```
%%writefile app.py
import streamlit as st

st.video("video.mp4")
```

```
Overwriting app.py
```





In [27]:
```
%%writefile app.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
import time

# Page Configuration
st.set_page_config(page_title="Student Marks Analyzer", layout="wide")

st.title("📊 Student Marks Analyzer")
st.markdown("Mini Project using **Streamlit + NumPy**")

# Sidebar Inputs
st.sidebar.header("Input Section")
rows = st.sidebar.number_input("Number of Students", 1, 10, 3)
cols = st.sidebar.number_input("Number of Subjects", 1, 6, 3)

# Generate Random Marks
if st.sidebar.button("Generate Marks"):
    with st.spinner("Generating data..."):
        time.sleep(1)
```

```
        marks = np.random.randint(30, 100, size=(rows, cols))
        st.success("Marks Generated Successfully")

    st.subheader("Generated Marks (2D NumPy Array)")
    st.write(marks)

    # Shape
    st.info(f"Array Shape: {marks.shape}")

    # Indexing & Slicing
    st.subheader("Indexing & Slicing")
    st.write("First Student Marks:", marks[0])
    st.write("First Subject Marks:", marks[:, 0])

    # Iteration
    st.subheader("Iteration")
    for i, student in enumerate(marks):
        st.write(f"Student {i+1} Marks:", student)

    # where()
    st.subheader("Students Passed (Marks >= 40)")
    passed = np.where(marks >= 40, "Pass", "Fail")
    st.write(passed)

    # sort()
    st.subheader("Sorted Marks")
    sorted_marks = np.sort(marks, axis=1)
    st.write(sorted_marks)

    # concatenate()
    st.subheader("Concatenated Array")
    bonus = np.full((rows, cols), 5)
    final_marks = np.concatenate((marks, bonus), axis=1)
    st.write(final_marks)

    # Charts
    st.subheader("Visualization")
    avg_marks = np.mean(marks, axis=1)

    col1, col2 = st.columns(2)

    with col1:
        st.line_chart(avg_marks)

    with col2:
        plt.bar(range(1, rows+1), avg_marks)
        st.pyplot(plt)

    # Expander
    with st.expander("Project Explanation"):
        st.write("This project demonstrates Streamlit UI components and NumPy array op

else:
    st.warning("Click 'Generate Marks' to start")
```

```
Overwriting app.py
```

https://drive.google.com/file/d/1Phdked2Q27x7aKimcwzRLk_KAG2AyUeI/view?usp=drive_link

Vishal Acharya

write a below python program for cricket analysis IN STREAMLIT

Class Data:

The Data class is an abstract base class (ABC) that defines a common interface for handling sports data. It includes the following class variables and methods:

Class Variables: .Array (Class Variable):

A class-level numpy array that holds the initial sports data.It represents scores of players in various matches, organized as a 2D arrayThere is an array of scores of 6 Batsmen in 5 T20 Matches. Which is given below.

Array=np.array([[64, 13, 100, 46, 69, 85], [54, 43, 7, 34, 17, 19], [73, 73, 90, 77, 53, 14], [74, 41, 41, 83, 74, 95], [48, 31, 74, 49, 67, 99]])

Abstract Method: abcmethod(self) (Abstract Method)

Class Match (Child/Sub class of Data): The Match class inherits from the abstract class Data and provides concrete implementations for its abstract methods. Additionally, it introduces specific methods for cricket match analysis. Let's provide detailed descriptions for the methods:

1. match_highscore(self, match_number): Finds and prints the maximum score in a specific match specified by match_number.

2. player_lowscore(self, player_number): Determines and prints the minimum score of a specific player across all matches.

3. match_total(self):

Adds an extra column to the existing array to represent the total score of each player across all matches. (Add the last column as the total scores column to the existing array and print the updated array.)

4. sorted_matchtotal_hightolow(self):

Sorts the array based on the total scores of players across all matches in descending order. Utilizes the match_total method to compute the total scores for each player. prints the sorted array.

5. player total score(self):

print the total scores of player

6. binary_array(self):

Create binary array if score>50 denote 1 or score<50 denote 2 7. lineplot (self, player1, player2): Generates a line plot showing the scores of two players across all matches. Retrieves the scores of the specified players from the array and plots them against the match numbers. 8.

Vishal Acharya

pieplot(self, match_number): Generates a pie chart showing the distribution of scores for a specific match.

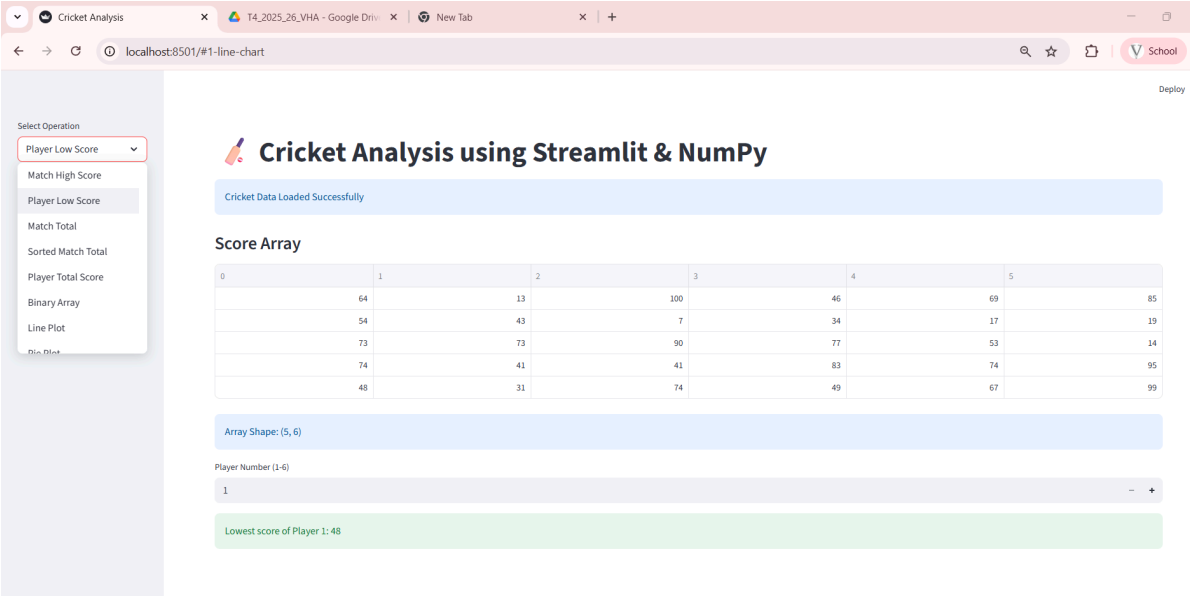
In [28]:
```
%%writefile app.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod

st.set_page_config(page_title="Cricket Analysis", layout="wide")
st.title("🏏 Cricket Analysis using Streamlit & NumPy")

# ---------------- ABSTRACT CLASS ----------------
class Data(ABC):
    Array = np.array([
        [64, 13, 100, 46, 69, 85],
        [54, 43, 7, 34, 17, 19],
        [73, 73, 90, 77, 53, 14],
        [74, 41, 41, 83, 74, 95],
        [48, 31, 74, 49, 67, 99]
    ])

    @abstractmethod
    def abcmethod(self):
        pass

# ---------------- CHILD CLASS ----------------
class Match(Data):
    def abcmethod(self):
        st.info("Cricket Data Loaded Successfully")

    def match_highscore(self, match_number):
        st.success(f"Highest score in Match {match_number}: {np.max(self.Array[match_n

    def player_lowscore(self, player_number):
        st.success(f"Lowest score of Player {player_number}: {np.min(self.Array[:, pla

    def match_total(self):
        total = np.sum(self.Array, axis=1).reshape(-1, 1)
        updated = np.concatenate((self.Array, total), axis=1)
        st.write(updated)
        return updated

    def sorted_matchtotal_hightolow(self):
        updated = self.match_total()
        sorted_array = updated[updated[:, -1].argsort()[::-1]]
        st.write(sorted_array)

    def player_total_score(self):
        totals = np.sum(self.Array, axis=1)
        for i, t in enumerate(totals):
            st.write(f"Player {i+1} Total Score: {t}")

    def binary_array(self):
        st.write(np.where(self.Array > 50, 1, 2))

    def lineplot(self, player1, player2):
        plt.figure()
        plt.plot(self.Array[:, player1-1], label=f"Player {player1}")
        plt.plot(self.Array[:, player2-1], label=f"Player {player2}")
        plt.xlabel("Matches")
```

Vishal Acharya

```
        plt.ylabel("Scores")
        plt.legend()
        st.pyplot(plt)

    def pieplot(self, match_number):
        plt.figure()
        plt.pie(self.Array[match_number-1],
                labels=[f"P{i+1}" for i in range(6)],
                autopct="%1.1f%%")
        plt.title(f"Match {match_number} Score Distribution")
        st.pyplot(plt)

# ---------------- STREAMLIT UI ----------------
match = Match()
match.abcmethod()

st.subheader("Score Array")
st.write(match.Array)
st.info(f"Array Shape: {match.Array.shape}")

option = st.sidebar.selectbox(
    "Select Operation",
    [
        "Match High Score",
        "Player Low Score",
        "Match Total",
        "Sorted Match Total",
        "Player Total Score",
        "Binary Array",
        "Line Plot",
        "Pie Plot"
    ]
)

if option == "Match High Score":
    m = st.number_input("Match Number (1-5)", 1, 5)
    match.match_highscore(m)
elif option == "Player Low Score":
    p = st.number_input("Player Number (1-6)", 1, 6)
    match.player_lowscore(p)
elif option == "Match Total":
    match.match_total()
elif option == "Sorted Match Total":
    match.sorted_matchtotal_hightolow()
elif option == "Player Total Score":
    match.player_total_score()
elif option == "Binary Array":
    match.binary_array()
elif option == "Line Plot":
    p1 = st.number_input("Player 1", 1, 6)
    p2 = st.number_input("Player 2", 1, 6)
    match.lineplot(p1, p2)
elif option == "Pie Plot":
    m = st.number_input("Match Number", 1, 5)
    match.pieplot(m)
```

```
Overwriting app.py
```

Vishal Acharya



Vishal Acharya

In [29]:

```
%%writefile app.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod

st.set_page_config(page_title="Cricket Analysis", layout="wide")
st.title("🏏 Cricket Analysis using Streamlit & NumPy")

# ---------------- ABSTRACT CLASS ----------------
class Data(ABC):
    Array = np.array([
        [64, 13, 100, 46, 69, 85],
        [54, 43, 7, 34, 17, 19],
        [73, 73, 90, 77, 53, 14],
        [74, 41, 41, 83, 74, 95],
        [48, 31, 74, 49, 67, 99]
    ])

    @abstractmethod
    def abcmethod(self):
        pass

# ---------------- CHILD CLASS ----------------
class Match(Data):
    def abcmethod(self):
        st.info("Cricket Data Loaded Successfully")

    def match_highscore(self, match_number):
        st.success(f"Highest score in Match {match_number}: {np.max(self.Array[match_n

    def player_lowscore(self, player_number):
        st.success(f"Lowest score of Player {player_number}: {np.min(self.Array[:, pla

    def match_total(self):
        total = np.sum(self.Array, axis=1).reshape(-1, 1)
        updated = np.concatenate((self.Array, total), axis=1)
        st.write(updated)
        return updated

    def sorted_matchtotal_hightolow(self):
        updated = self.match_total()
```

Vishal Acharya

```
        sorted_array = updated[updated[:, -1].argsort()[::-1]]
        st.write(sorted_array)

    def player_total_score(self):
        totals = np.sum(self.Array, axis=1)
        for i, t in enumerate(totals):
            st.write(f"Player {i+1} Total Score: {t}")

    def binary_array(self):
        st.write(np.where(self.Array > 50, 1, 2))

    def lineplot(self, player1, player2):
        plt.figure(figsize=(4, 3))
        plt.plot(self.Array[:, player1-1], label=f"Player {player1}")
        plt.plot(self.Array[:, player2-1], label=f"Player {player2}")
        plt.xlabel("Matches")
        plt.ylabel("Scores")
        plt.legend()
        st.pyplot(plt)

    def pieplot(self, match_number):
        plt.figure(figsize=(4, 4))
        plt.pie(
            self.Array[match_number-1],
            labels=[f"P{i+1}" for i in range(6)],
            autopct="%1.1f%%"
        )
        plt.title(f"Match {match_number} Score Distribution")
        st.pyplot(plt)

# ---------------- STREAMLIT OUTPUT ----------------
match = Match()
match.abcmethod()

st.subheader("Score Array")
st.write(match.Array)
st.info(f"Array Shape: {match.Array.shape}")

# -------- Inputs Only --------
st.subheader("User Inputs")
match_no = st.number_input("Enter Match Number (1-5)", 1, 5, 1)
player_no = st.number_input("Enter Player Number (1-6)", 1, 6, 1)
player1 = st.number_input("Line Plot Player 1", 1, 6, 1)
player2 = st.number_input("Line Plot Player 2", 1, 6, 2)

# -------- All Outputs Displayed --------
st.subheader("Match High Score")
match.match_highscore(match_no)

st.subheader("Player Low Score")
match.player_lowscore(player_no)

st.subheader("Match Total (with Total Column)")
match.match_total()

st.subheader("Sorted Match Total (High to Low)")
match.sorted_matchtotal_hightolow()

st.subheader("Player Total Scores")
match.player_total_score()

st.subheader("Binary Array (Score > 50 → 1 else 2)")
```

```
match.binary_array()

st.subheader("Line Plot (Two Players)")
match.lineplot(player1, player2)

st.subheader("Pie Chart (Selected Match)")
match.pieplot(match_no)
```

```
Overwriting app.py
```

https://drive.google.com/file/d/1cUHF-TVwERX6wCHn5Yki0twafhU7JLB0/view?usp=drive_link

In [30]:

```
%%writefile app.py
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
from abc import ABC, abstractmethod

st.set_page_config(page_title="Cricket Analysis (NumPy Only)", layout="wide")
st.title("🏏 Cricket Analysis using Streamlit & NumPy (Syllabus Only)")

# ---------------- ABSTRACT CLASS ----------------
class Data(ABC):
    # 2D Array creation
    Array = np.array([
        [64, 13, 100, 46, 69, 85],
        [54, 43, 7, 34, 17, 19],
        [73, 73, 90, 77, 53, 14],
        [74, 41, 41, 83, 74, 95],
        [48, 31, 74, 49, 67, 99]
    ])

    @abstractmethod
    def abcmethod(self):
        pass


# ---------------- CHILD CLASS ----------------
class Match(Data):
    def abcmethod(self):
        st.info("Cricket Data Loaded Successfully")

    # -------- Indexing + Iteration --------
    def match_highscore(self, match_number):
        row = self.Array[match_number - 1]   # indexing
        high = row[0]
        for val in row:                      # iteration
            if val > high:
                high = val
        st.success(f"Highest score in Match {match_number}: {high}")

    # -------- Slicing + Iteration --------
    def player_lowscore(self, player_number):
        col = self.Array[:, player_number - 1]   # slicing
        low = col[0]
        for val in col:
            if val < low:
                low = val
        st.success(f"Lowest score of Player {player_number}: {low}")

    # -------- Iteration + Reshape + Concatenate --------
    def match_total(self):
```

Vishal Acharya

Vishal Acharya

```python
        totals = []
        for row in self.Array:
            s = 0
            for val in row:
                s += val
            totals.append(s)

        totals = np.array(totals).reshape(5, 1)   # reshape
        updated = np.concatenate((self.Array, totals), axis=1)
        st.write(updated)
        return updated

    # -------- Sort (only np.sort) --------
    def sorted_matchtotal_hightolow(self):
        updated = self.match_total()
        totals = updated[:, -1]            # slicing
        sorted_totals = np.sort(totals)    # sort

        sorted_rows = []
        for t in sorted_totals[::-1]:      # iteration
            for row in updated:
                if row[-1] == t and list(row) not in sorted_rows:
                    sorted_rows.append(list(row))

        st.write(np.array(sorted_rows))

    # -------- Iteration --------
    def player_total_score(self):
        for i in range(self.Array.shape[1]):    # shape
            s = 0
            for val in self.Array[:, i]:
                s += val
            st.write(f"Player {i+1} Total Score: {s}")

    # -------- where --------
    def binary_array(self):
        binary = np.where(self.Array > 50, 1, 2)
        st.write(binary)

    # -------- Indexing + Slicing --------
    def lineplot(self, player1, player2):
        plt.figure(figsize=(4, 3))
        plt.plot(self.Array[:, player1-1], label=f"Player {player1}")
        plt.plot(self.Array[:, player2-1], label=f"Player {player2}")
        plt.xlabel("Matches")
        plt.ylabel("Scores")
        plt.legend()
        st.pyplot(plt)

    # -------- array_split --------
    def pieplot(self, match_number):
        split_matches = np.array_split(self.Array, 5)  # array_split
        match_data = split_matches[match_number - 1][0]

        plt.figure(figsize=(4, 4))
        plt.pie(
            match_data,
            labels=[f"P{i+1}" for i in range(6)],
            autopct="%1.1f%%"
        )
        plt.title(f"Match {match_number} Score Distribution")
        st.pyplot(plt)
```

Vishal Acharya

```python
# ---------------- STREAMLIT OUTPUT ----------------
match = Match()
match.abcmethod()

st.subheader("Score Array")
st.write(match.Array)

st.subheader("Array Shape")
st.write(match.Array.shape)

# -------- Inputs --------
st.subheader("User Inputs")
match_no = st.number_input("Enter Match Number (1-5)", 1, 5, 1)
player_no = st.number_input("Enter Player Number (1-6)", 1, 6, 1)
player1 = st.number_input("Line Plot Player 1", 1, 6, 1)
player2 = st.number_input("Line Plot Player 2", 1, 6, 2)

# -------- All Outputs --------
st.subheader("Match High Score")
match.match_highscore(match_no)

st.subheader("Player Low Score")
match.player_lowscore(player_no)

st.subheader("Match Total (Concatenate)")
match.match_total()

st.subheader("Sorted Match Total (High → Low)")
match.sorted_matchtotal_hightolow()

st.subheader("Player Total Scores")
match.player_total_score()

st.subheader("Binary Array (where)")
match.binary_array()

st.subheader("Line Plot")
match.lineplot(player1, player2)

st.subheader("Pie Chart")
match.pieplot(match_no)
```

```
Overwriting app.py
```

https://drive.google.com/file/d/1uXF8eHhdofLgmDZt4_siZcKpfdZ2LwaJ/view?usp=drive_link